Personal Learning Journey- IT3091 Machine Learning - Group 2026-AI-05

DataCo Smart Supply Chain: late-delivery risk

Name: Jayasekara C.D.

IT number: IT24103494

Area of responsibility: Exploratory data analysis and hyperparameter tuning

My role

I contributed to the exploratory data analysis and then owned the alternative models and the hyperparameter tuning. My job was to find out whether anything more complicated than the baseline actually helps.

What I did

On the EDA side I worked on the feature importance and mutual information analysis, and on the comparison of delay rates across product categories and departments.

For modelling I trained four alternatives: a Decision Tree, a Random Forest, XGBoost and a small neural network. I then tuned the Random Forest with a randomised search over twelve candidate settings, scored on ROC-AUC, using the same order-aware cross-validation as the rest of the project.

What I learned

The main thing I learned was how to distinguish real improvements from noise. The tuned Random Forest score increased from 0.7747 to 0.7760, but the 0.0012 gain was smaller than the fold variation of 0.0023, so the improvement wasn't significant.

I also understood bagging and boosting in practice. Random Forest averages independent trees to reduce variance, while XGBoost builds trees sequentially to correct previous errors and reduce bias. The neural network scored 0.7407, below the Logistic Regression baseline, showing that a more complex model isn't always better.

What I found difficult

Accepting a small improvement was difficult because I expected tuning to produce better results. It was tempting to expand the search until the score improved, but I realised this could lead to overfitting the validation set. I also had trouble setting up the custom cross-validation splitter because it required the order groups to be passed in explicitly.

A mistake I caught

After I added the tuned model, the comparison table still pointed at the untuned one, so the threshold analysis and the final evaluation were silently using the wrong model. I noticed because the final test number did not match the tuned model's validation score, and I added a step to rebuild the comparison table after tuning.

What I would do differently

I would run the cheaper models first and use their spread to decide whether tuning was worth doing at all. By the time I tuned, the evidence that the models were tied was already there, and that would have told me what to expect.

On using AI

I used Claude for the search setup and for explanations of the hyperparameters. It gave a sensible starting search space, but it also suggested a cross-validation approach that would not have worked with our data ordering, which we caught by checking rather than assuming.